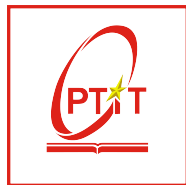


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN CÁ NHÂN

PHÁT TRIỂN ỨNG DỤNG CHO
THIẾT BỊ DI ĐỘNG (MAD)

PHÂN HỆ API GATEWAY & IDENTITY
SERVICE

Hạ tầng kết nối và Quản lý danh tính người dùng ứng dụng Pody

Giảng viên hướng dẫn: *ThS. Nguyễn Hoàng Anh*

Nhóm BTL: *01*

Sinh viên thực hiện: *Nguyễn Việt Huy – B22DCCN393*

Nhóm thành viên BTL chính:

STT	Họ và tên	MSSV	Phân công chính
1	Lại Xuân Hiếu	B22DCCN309	AI podcast (ai-service)
2	Vũ Đức Thành	B22DCCN801	Báo & Embedding (article, embedding)
3	Nguyễn Việt Huy	B22DCCN393	Hạ tầng & Auth (gateway, identity)
4	Lê Minh Ngọc	B22DCCN586	Thông báo (notification-service)

Hà Nội, 2026

MỤC LỤC

1	DANH SÁCH CHỨC NĂNG ĐƯỢC PHÂN CÔNG	5
1.1	Phân hệ API Gateway (Cổng kết nối hệ thống)	5
1.2	Phân hệ Identity Service (Quản lý danh tính người dùng)	6
2	KIẾN TRÚC CHI TIẾT HỆ THỐNG	7
2.1	Mô hình thành phần chi tiết (Component Diagram)	7
2.2	Cơ chế kết nối và tương tác giữa các thành phần	9
2.2.1	Tương tác đồng bộ (Synchronous Interaction via HTTP/REST)	9
2.2.2	Tương tác bất đồng bộ (Asynchronous Interaction via Transactional Outbox & Kafka)	10
3	MÃ NGUỒN VÀ DỮ LIỆU ĐÁP ỨNG CHỨC NĂNG	11
3.1	Cấu trúc các bảng dữ liệu trong CSDL (Database Schemas)	11
3.1.1	Bảng users	12
3.1.2	Bảng user_identities	12
3.1.3	Bảng auth_sessions	13
3.1.4	Bảng email_verification_tokens	13
3.1.5	Bảng password_reset_tokens	14
3.1.6	Bảng outbox_events	14
3.2	Bản đồ ánh xạ mã nguồn đáp ứng chức năng (Code Mapping)	15
3.3	Tích hợp với dịch vụ bên thứ ba (External APIs)	16
3.3.1	Google OAuth2 API (https://oauth2.googleapis.com/tokeninfo)	16
3.3.2	Supabase Storage REST API	16
3.4	Sơ đồ trình tự nghiệp vụ (Sequence Diagrams)	16
4	HƯỚNG DẪN VÀ CÁC LƯU Ý KHI CÀI ĐẶT TRIỂN KHAI	19
4.1	Yêu cầu môi trường tối thiểu	19
4.2	Quy trình các bước triển khai chi tiết	19
4.2.1	Bước 1: Clone mã nguồn dự án	20
4.2.2	Bước 2: Thiết lập cấu hình môi trường	20
4.2.3	Bước 3: Khởi chạy tầng hạ tầng cơ sở (Infrastructure)	20

4.2.4	Bước 4: Thực thi di trú cơ sở dữ liệu (Database Migrations)	20
4.2.5	Bước 5: Biên dịch và Khởi chạy các Dịch vụ Nghiệp vụ (Microservices)	21
4.3	Các lưu ý quan trọng và Kinh nghiệm xử lý sự cố (Troubleshooting)	21
4.3.1	Đồng bộ trạng thái khởi động (Service Startup Ordering)	21
4.3.2	Kích hoạt và Kiểm thử Trang cầu nối Deep Link trên Di động	21
4.3.3	Sự cố nghẽn tiến trình Transactional Outbox	22
4.3.4	Xác thực nhanh trạng thái hoạt động của hệ thống	22

DANH SÁCH HÌNH VẼ

2.1	Sơ đồ Component phân hệ API Gateway và Identity Service . . .	8
3.1	Sơ đồ thực thể liên kết (ER) của Identity Database	11
3.2	Sơ đồ trình tự luồng đăng ký tài khoản và xác minh email	17
3.3	Sơ đồ trình tự luồng gọi API bảo vệ qua API Gateway	18

DANH SÁCH BẢNG

3.1	Bản đồ ánh xạ các chức năng và mã nguồn liên quan	15
4.1	Bảng URL kiểm thử nhanh hệ thống	23

CHƯƠNG 1

DANH SÁCH CHỨC NĂNG ĐƯỢC PHÂN CÔNG

Trong phạm vi đề tài bài tập lớn, phân hệ **API Gateway** và **Identity Service** được phân công thực hiện với các chức năng chi tiết như sau:

1.1 Phân hệ API Gateway (Cổng kết nối hệ thống)

- **Định tuyến động (Dynamic Routing):** Xây dựng cổng truy cập HTTP tập trung tại cổng 8080, cấu hình reverse proxy chuyển tiếp yêu cầu đến các microservices nội bộ tương ứng dựa trên tiền tố của đường dẫn (Path Prefix).
- **Cơ chế phân tách phân quyền (Public/Protected Routes):** Thiết lập bộ lọc router để phân tách rõ ràng các API công khai cho phép khách truy cập tự do (/api/v1/public/*) và các API bảo mật (/api/v1/*) yêu cầu người dùng phải xác thực.
- **Xác thực mã thông báo JWT (JWT Authentication Middleware):** Thiết lập bộ lọc chặn (intercept) tất cả request gửi vào các protected routes, giải mã và kiểm tra tính hợp lệ chữ ký của Access Token gửi kèm ở header.
- **Làm phong phú thông tin Request (Header Enrichment):** Trích xuất định danh người dùng (user_id) và quyền hạn từ token đã giải mã, đính kèm vào tiêu đề HTTP (X-Auth-User-ID) để gửi sang các microservices downstream.
- **Cơ chế ghi nhận và truy vết (Request Tracing):** Tự động sinh mã duy nhất Request-ID cho mỗi yêu cầu kết nối, ghi log tập trung để hỗ trợ giám sát và gỡ lỗi xuyên suốt hệ thống microservices.
- **Hỗ trợ Trang cầu nối ứng dụng di động (Deep Link Bridges):** Tạo các trang web HTML trung gian tĩnh tại Gateway để xử lý điều hướng

sâu (Deep Linking) cho các tác vụ kích hoạt tài khoản hoặc đặt lại mật khẩu từ liên kết email chuyên hướng vào ứng dụng Mobile.

1.2 Phân hệ Identity Service (Quản lý danh tính người dùng)

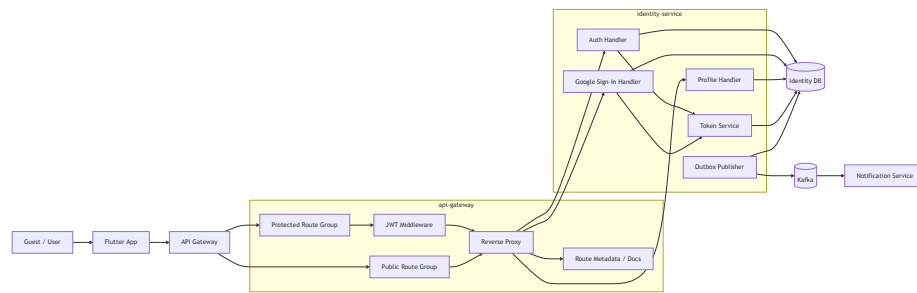
- **Đăng ký tài khoản (User Sign-up):** Cho phép đăng ký bằng email và mật khẩu. Mã hóa một chiều mật khẩu bằng thuật toán bcrypt có độ muối cao trước khi lưu vào CSDL.
- **Xác minh tài khoản qua Email (Email Verification):** Sinh mã token ngẫu nhiên dạng mã băm bảo mật gửi qua hộp thư điện tử của người dùng. Kích hoạt tài khoản khi mã khớp và còn hiệu lực.
- **Đăng nhập truyền thống (Email/Password Sign-in):** Kiểm tra thông tin xác thực, sinh và trả về bộ đôi mã thông báo Access Token (thời hạn 15 phút) và Refresh Token (thời hạn 30 ngày).
- **Đăng nhập liên kết bên thứ ba (Google Sign-In OAuth2):** Tiếp nhận ID Token từ client, tích hợp trực tiếp với dịch vụ Google API để kiểm chứng danh tính của người dùng, tự động khởi tạo hồ sơ nếu là lần đầu truy cập.
- **Quản lý phiên làm việc và Làm mới phiên (Refresh Token Rotation):** Lưu thông tin băm của Refresh Token vào bảng quản lý phiên hoạt động. Hỗ trợ cơ chế luân phiên cấp mới mã thông báo để bảo vệ chống giả mạo phiên.
- **Khôi phục và Đặt lại mật khẩu (Forgot/Reset Password):** Tiếp nhận yêu cầu quên mật khẩu, sinh mã xác thực OTP dùng một lần (One-Time Password) gửi qua email, kiểm tra tính hợp lệ và cho phép đặt lại mật khẩu mới đồng thời thu hồi mọi phiên làm việc hiện hành.
- **Quản lý thông tin hồ sơ và Ảnh đại diện (Profile & Avatar Upload):** Cung cấp API cập nhật thông tin cá nhân của người dùng hiện tại, tích hợp với API ngoài của dịch vụ Supabase Storage để lưu trữ và lấy đường dẫn ảnh đại diện.

CHƯƠNG 2

KIẾN TRÚC CHI TIẾT HỆ THỐNG

2.1 Mô hình thành phần chi tiết (Component Diagram)

Kiến trúc của phân hệ hạ tầng được thiết kế nhằm mục đích đảm bảo tính độc lập giữa các thành phần nghiệp vụ, giảm thiểu sự liên kết cứng (loose coupling) và tối ưu hóa hiệu năng phản hồi. Sơ đồ các thành phần chi tiết của API Gateway và Identity Service cùng các mối quan hệ được biểu diễn trong Hình 2.1.



Hình 2.1: Sơ đồ Component phân hệ API Gateway và Identity Service

Các thành phần cấu thành chính bao gồm:

- **Flutter Mobile Client:** Điểm đầu tiếp xúc với người dùng, kết nối tới API Gateway qua giao thức HTTP REST.
- **API Gateway (Go / Chi Router):** Nhận request, xử lý CORS, phân luồng theo route, kiểm tra chữ ký token JWT qua middleware. Sử dụng ReverseProxy của ngôn ngữ Go để chuyển tiếp yêu cầu đến các microservices nằm trong mạng nội bộ.
- **Identity Service (Go):** Nhận request nội bộ, thực thi logic đăng ký, đăng

nhập, quản lý phiên và hồ sơ cá nhân.

- **Identity Database (PostgreSQL):** Hệ quản trị CSDL quan hệ lưu trữ độc lập dữ liệu người dùng, token và phiên làm việc.
- **Apache Kafka Message Broker:** Đóng vai trò là kênh truyền tin bất đồng bộ. Identity Service sử dụng cơ chế Transactional Outbox ghi tin nhắn vào DB rồi đẩy lên Kafka nhằm tránh việc thất thoát sự kiện khi xảy ra lỗi mạng.

2.2 Cơ chế kết nối và tương tác giữa các thành phần

Sự tương tác giữa các thành phần diễn ra qua hai hình thức chính:

2.2.1 Tương tác đồng bộ (Synchronous Interaction via HTTP/REST)

Sử dụng cho các thao tác yêu cầu kết quả phản hồi tức thời từ hệ thống (Đăng ký, Đăng nhập, Truy vấn Profile, Đổi avatar).

1. Client gửi HTTP request chứa Access Token trong header Authorization: Bearer <token> đến Gateway.
2. Gateway chặn request, kiểm tra định dạng và verify chữ ký số của token bằng khóa bí mật chung (JWT_SECRET). Nếu không hợp lệ, trả ngay mã lỗi 401 Unauthorized.
3. Nếu token hợp lệ, Gateway trích xuất trường user_id từ token payload, ghi đè vào request header X-Auth-User-ID và chuyển tiếp (forward) request đến cổng nội bộ của identity-service.
4. identity-service nhận request, đọc thông tin từ header X-Auth-User-ID để định danh người dùng và truy vấn thông tin cần thiết từ Identity Database để trả về kết quả.

2.2.2 Tương tác bất đồng bộ (Asynchronous Interaction via Transactional Outbox & Kafka)

Sử dụng cho các hành vi nghiệp vụ cần thông báo cho phân hệ khác xử lý mà không cần client phải đợi phản hồi tức thời (gửi email xác minh tài khoản, gửi OTP khôi phục mật khẩu).

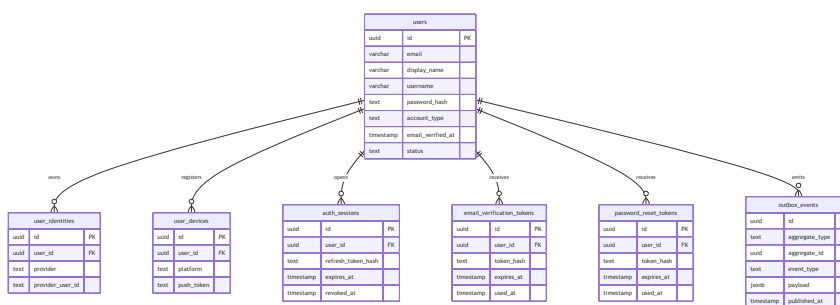
1. **Lưu trữ giao dịch nhất quán:** Khi có sự kiện (ví dụ: đăng ký tài khoản), trong cùng một Database Transaction, `identity-service` tiến hành chèn thông tin user mới vào bảng `users` và chèn bản ghi mô tả sự kiện vào bảng `outbox_events`.
2. **Đẩy tin nhắn ngầm (Outbox Publisher):** Một tiến trình chạy nền độc lập (`OutboxPublisher`) thực hiện quét bảng `outbox_events` định kỳ với chu kỳ ngắn (100ms) để lấy các sự kiện chưa gửi, đóng gói và phát (publish) lên Kafka topic `identity-events`.
3. **Dọn dẹp sự kiện đã gửi (Outbox Cleanup):** Sau khi nhận được tín hiệu phản hồi xác nhận gửi thành công từ Kafka broker, bản ghi sự kiện trong `outbox` được đánh dấu là đã gửi thành công. Tiến trình `OutboxCleanup` chạy ngầm định kỳ quét và xóa các bản ghi sự kiện đã gửi thành công cũ để làm gọn CSDL.
4. **Tiêu thụ sự kiện downstream (Notification Consumer):** Phân hệ `notification service` lắng nghe topic Kafka này, đọc thông tin sự kiện, gọi dịch vụ gửi thư SMTP để chuyển email kích hoạt hoặc OTP đến hộp thư của người dùng.

CHƯƠNG 3

MÃ NGUỒN VÀ DỮ LIỆU ĐÁP ỨNG CHỨC NĂNG

3.1 Cấu trúc các bảng dữ liệu trong CSDL (Database Schemas)

Identity DB sử dụng hệ quản trị cơ sở dữ liệu PostgreSQL. Sơ đồ thực thể liên kết (ER Diagram) cho các bảng thuộc phân hệ được biểu diễn trong Hình 3.1.



Hình 3.1: Sơ đồ thực thể liên kết (ER) của Identity Database

Chi tiết cấu trúc và ý nghĩa của các bảng cơ sở dữ liệu như sau:

3.1.1 Bảng users

Lưu trữ thông tin định danh gốc của người dùng hệ thống.

- **id:** UUID (Primary Key) – Khóa chính tự sinh định danh duy nhất của người dùng.
- **email:** VARCHAR(255) (Unique) – Địa chỉ thư điện tử dùng để đăng nhập.
- **username:** VARCHAR(50) (Unique, Nullable) – Tên tài khoản hiển thị hệ thống.
- **password_hash:** VARCHAR(255) (Nullable) – Mật khẩu đã được mã hóa băm bcrypt một chiều. Trường này rỗng nếu tài khoản đăng nhập trực tiếp qua liên kết Google.
- **display_name:** VARCHAR(100) – Tên hiển thị công khai trên ứng dụng.
- **avatar_url:** VARCHAR(500) (Nullable) – Liên kết đến tệp ảnh đại diện được lưu trữ trên Supabase Storage.
- **status:** VARCHAR(20) – Trạng thái tài khoản (ví dụ: pending_verification, active, suspended).
- **account_type:** VARCHAR(20) – Loại tài khoản (email, google).
- **locale:** VARCHAR(5) – Thiết lập ngôn ngữ mặc định (vi, en).
- **email_verified_at:** TIMESTAMP – Thời gian xác minh thư điện tử thành công.

3.1.2 Bảng user_identities

Quản lý các tài khoản liên kết đăng nhập bên thứ ba (Google OAuth2).

- **id:** BIGSERIAL (Primary Key) – Khóa chính tự tăng.

- `user_id`: UUID (Foreign Key) – Tham chiếu liên kết đến bảng `users.id`.
- `provider`: VARCHAR(50) – Tên nhà cung cấp xác thực (google).
- `provider_user_id`: VARCHAR(255) – Khóa ID định danh duy nhất của người dùng do Google cung cấp.

3.1.3 Bảng `auth_sessions`

Quản lý các phiên đăng nhập đang hoạt động và hỗ trợ thu hồi quyền truy cập (Sign-out / Revoke session).

- `id`: UUID (Primary Key) – Mã định danh phiên làm việc.
- `user_id`: UUID (Foreign Key) – Tham chiếu liên kết đến bảng `users.id`.
- `refresh_token_hash`: VARCHAR(255) – Mã băm bảo mật SHA256 của Refresh Token tương ứng nhằm bảo vệ chống lộ mã thông báo dưới dạng văn bản rõ.
- `expires_at`: TIMESTAMP – Thời hạn phiên đăng nhập hết hiệu lực.
- `is_revoked`: BOOLEAN – Trạng thái đã thu hồi phiên truy cập (mặc định là `false`).

3.1.4 Bảng `email_verification_tokens`

Lưu giữ mã băm dùng để xác thực hòm thư điện tử.

- `id`: BIGSERIAL (Primary Key) – Khóa chính.
- `user_id`: UUID (Foreign Key) – Liên kết đến bảng `users.id`.
- `token_hash`: VARCHAR(255) – Mã xác thực băm SHA256 gửi qua email.
- `expires_at`: TIMESTAMP – Hạn hết hiệu lực của mã xác thực (24 giờ).

3.1.5 Bảng password_reset_tokens

Lưu trữ mã OTP bảo mật để phục vụ khôi phục đặt lại mật khẩu.

- `id`: BIGSERIAL (Primary Key) – Khóa chính.
- `user_id`: UUID (Foreign Key) – Liên kết đến bảng `users.id`.
- `token_hash`: VARCHAR(255) – Mã OTP bấm dùng một lần.
- `expires_at`: TIMESTAMP – Thời điểm mã OTP hết hạn (15 phút).

3.1.6 Bảng outbox_events

Lưu trữ tạm các sự kiện phát sinh từ hệ thống Identity trước khi đẩy lên Kafka.

- `id`: UUID (Primary Key) – Khóa chính định danh sự kiện.
- `aggregate_type`: VARCHAR(50) – Loại thực thể gây ra sự kiện (user).
- `aggregate_id`: VARCHAR(255) – Khóa định danh của thực thể liên quan (`user_id`).
- `event_type`: VARCHAR(100) – Tên sự kiện (ví dụ: `UserRegistered`, `PasswordResetRequested`, `UserUpdated`).
- `payload`: JSONB – Nội dung dữ liệu chi tiết của sự kiện.
- `status`: VARCHAR(20) – Trạng thái gửi tin (ví dụ: `pending`, `published`, `failed`).
- `attempts`: INTEGER – Số lần đã thử gửi tin nhắn lên broker.
- `last_error`: TEXT – Nội dung lỗi kỹ thuật ở lần thử gửi gần nhất.

3.2 Bản đồ ánh xạ mã nguồn đáp ứng chức năng (Code Mapping)

Toàn bộ mã nguồn backend của cụm Gateway và Identity Service được viết bằng ngôn ngữ lập trình Go. Bảng 3.1 mô tả cụ thể sự liên kết giữa các chức năng nghiệp vụ với các tệp tin mã nguồn, cấu trúc dữ liệu và các hàm tương ứng.

Bảng 3.1: Bản đồ ánh xạ các chức năng và mã nguồn liên quan

Chức năng phân công	Lớp / Hàm nghiệp vụ	Tệp nguồn liên quan và Giải thích
Reverse Proxy	<code>httpserver.newServiceProxy</code> <code>httpserver.joinPaths</code>	<code>api-gateway/internal/httpserver/proxy.go</code> Định tuyến và proxy request, xử lý ghi đè và cắt bỏ tiền tố API để chuyển đổi URL trước khi gọi hạ tầng downstream.
Xác thực JWT	<code>httpserver.withJWTAuth</code> <code>httpserver.enrichRequestWithClaims</code>	<code>api-gateway/internal/httpserver/middleware.go</code> Bộ lọc middleware giải mã token JWT bằng khóa bí mật chung, kiểm tra chữ ký và trích xuất claims gắn vào request headers.
Đăng ký tài khoản	<code>AuthService.SignUp</code> <code>PostgresStore.CreateUserWithEmail</code>	<code>identity-service/internal/auth/service.go</code> <code>identity-service/internal/store/postgres.go</code> Mã hóa bcrypt mật khẩu, chèn bản ghi user trạng thái chờ xác minh kèm tạo outbox event trong cùng một Transaction CSDL.
Đăng nhập tài khoản	<code>AuthService.SignIn</code> <code>PostgresStore.CreateSession</code>	<code>identity-service/internal/auth/service.go</code> Đối sánh mật khẩu đã băm, cấp bộ Access/Refresh Token và lưu hash Refresh Token vào bảng session để theo dõi.
Google Sign-In	<code>AuthService.SignInWithGoogle</code> <code>googleauth.VerifyToken</code>	<code>identity-service/internal/auth/service.go</code> <code>identity-service/internal/googleauth/google.go</code> Gọi API ngoài của Google để xác thực ID Token nhận từ ứng dụng di động, đồng bộ thông tin tài khoản người dùng tương ứng.
Quản lý Ảnh đại diện	<code>httpserver.server.updateAvatar</code> <code>storage.SupabaseStorage.UploadAvatar</code>	<code>identity-service/internal/httpserver/server.go</code> <code>identity-service/internal/media/supabase.go</code> Nhận file ảnh tải lên, tích hợp API ngoài của Supabase qua HTTP Client tải ảnh lên Bucket và cập nhật URL ảnh vào CSDL.
Làm mới phiên	<code>AuthService.RefreshToken</code> <code>tokens.GenerateTokenPair</code>	<code>identity-service/internal/auth/service.go</code> Verify Refresh Token gửi từ client, thu hồi phiên đăng nhập cũ trong CSDL và phát hành bộ mã thông báo mới thay thế.
Quên & Đặt lại mật khẩu	<code>AuthService.ForgotPassword</code> <code>AuthService.ResetPassword</code>	<code>identity-service/internal/auth/service.go</code> ForgotPassword sinh mã OTP gửi qua outbox. ResetPassword kiểm tra mã OTP, cập nhật mật khẩu đã băm bcrypt và revoke mọi phiên làm việc cũ.
Đẩy tin nhắn ngầm	<code>outbox.Publisher.Start</code> <code>outbox.Cleanup.Start</code>	<code>identity-service/internal/outbox/publisher.go</code> Tiến trình chạy ngầm quét bảng outbox định kỳ, đẩy dữ liệu JSON sự kiện lên Kafka Broker và thực hiện dọn dẹp các bản ghi cũ.

3.3 Tích hợp với dịch vụ bên thứ ba (External APIs)

Phân hệ hạ tầng tích hợp hai API bên thứ ba để đảm bảo các tính năng hoạt động an toàn và phong phú:

3.3.1 Google OAuth2 API (<https://oauth2.googleapis.com/tokeninfo>)

- **Mục đích:** Hỗ trợ tính năng đăng nhập nhanh thông qua tài khoản Google bằng cách kiểm thực tính hợp lệ của thẻ định danh nhận được từ Client.
- **Cơ chế gọi:** Sử dụng thư viện HTTP client chuẩn của Go để tạo yêu cầu GET gửi lên API của Google kèm theo tham số truy vấn là ID Token của người dùng. Google API trả về cấu trúc JSON chứa các trường email, name, picture và sub (mã định danh người dùng duy nhất của Google). Hệ thống dựa vào đó để liên kết tài khoản.

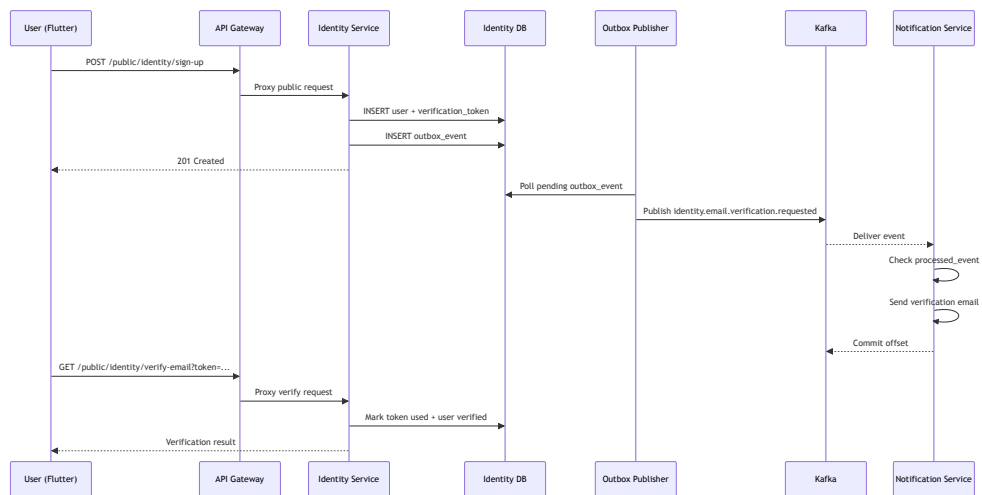
3.3.2 Supabase Storage REST API

- **Mục đích:** Lưu trữ tập tin ảnh đại diện của người dùng công khai trên Internet.
- **Cơ chế gọi:** Sử dụng HTTP REST Client gửi request POST có định dạng payload là multipart/form-data đến endpoint lưu trữ của Supabase (/storage/v1/object/avatars/), đính kèm mã xác thực Authorization Bearer chứa API Key của dự án để tải file lên bucket, sau đó lấy về URL liên kết tĩnh để cập nhật vào cơ sở dữ liệu.

3.4 Sơ đồ trình tự nghiệp vụ (Sequence Diagrams)

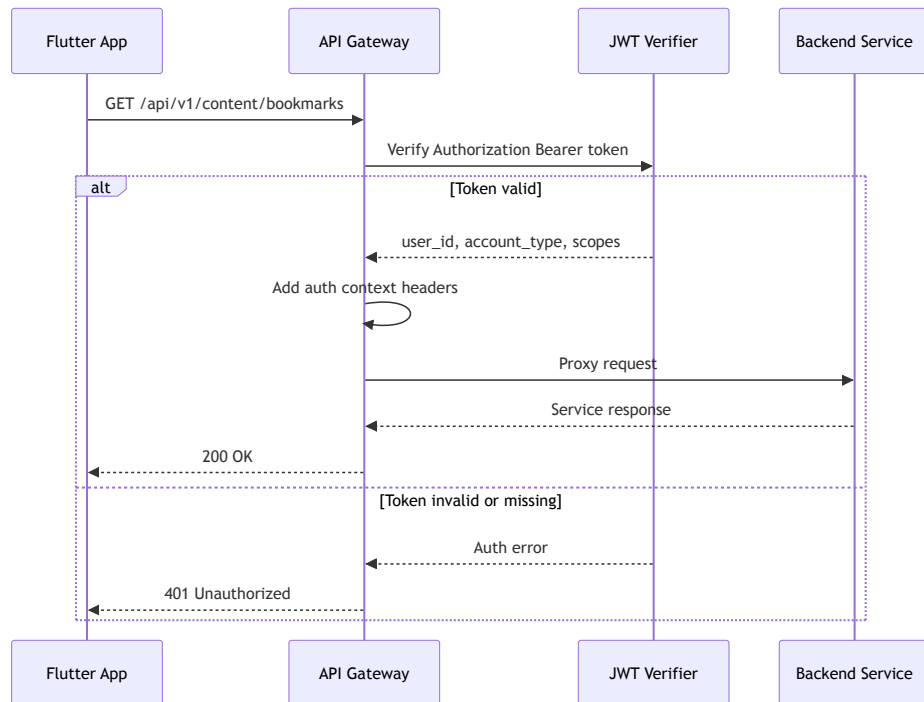
Sự tương tác chi tiết giữa các phân hệ và thành phần mã nguồn được thể hiện rõ nét qua các sơ đồ trình tự dưới đây.

Hình 3.2 mô tả chi tiết quy trình đăng ký tài khoản mới và xác minh qua thư điện tử. Có sự tham gia của Transactional Outbox để đảm bảo tính nhất quán dữ liệu giữa DB và Kafka:



Hình 3.2: Sơ đồ trình tự luồng đăng ký tài khoản và xác minh email

Hình 3.3 mô tả quá trình API Gateway đóng vai trò là chốt chặn bảo mật, tiếp nhận các yêu cầu truy cập được bảo vệ và ủy quyền chuyển tiếp dữ liệu đến các service downstream:



Hình 3.3: Sơ đồ trình tự luồng gọi API bảo vệ qua API Gateway

CHƯƠNG 4

HƯỚNG DẪN VÀ CÁC LƯU Ý KHI CÀI ĐẶT TRIỂN KHAI

4.1 Yêu cầu môi trường tối thiểu

Để đảm bảo triển khai và vận hành hệ thống cục bộ (local) trơn tru, máy chủ phát triển cần chuẩn bị sẵn sàng cấu hình sau:

- **Phần cứng:** CPU tối thiểu 4 Cores, bộ nhớ trong RAM tối thiểu 16GB (khuyến nghị 24GB do chạy đồng thời nhiều database Postgres độc lập và Apache Kafka Broker).
- **Phần mềm:** Docker Desktop (phiên bản v4.20 trở lên) cài kèm Docker Compose CLI, Git Client.
- **Cổng mạng tự do:** Đảm bảo các cổng kết nối dịch vụ sau đây trên máy host chưa bị ứng dụng nào khác chiếm dụng trước khi bật container:
 - 8080: API Gateway (Cổng giao tiếp duy nhất ra ngoài).
 - 8081: Identity Service API nội bộ.
 - 5433: PostgreSQL instance chính (Article DB).
 - 5434: PostgreSQL instance phụ (Embedding DB).
 - 6379: Redis Cache.
 - 8090: Giao diện quản trị Kafka UI.
 - 9000 & 9001: MinIO Console & Storage API.

4.2 Quy trình các bước triển khai chi tiết

Hệ thống Pody cung cấp các tệp Docker Compose cấu trúc riêng biệt để đơn giản hóa quá trình khởi chạy. Các bước thực hiện tuần tự như sau:

4.2.1 Bước 1: Clone mã nguồn dự án

Truy cập Terminal hoặc PowerShell, chạy lệnh sau để tải mã nguồn dự án về thư mục làm việc:

```
git clone -b main https://github.com/pody-team/pody-app.git
cd pody-app
```

4.2.2 Bước 2: Thiết lập cấu hình môi trường

Tạo tệp cấu hình môi trường .env tại thư mục gốc của dự án. Khai báo các khóa bí mật cần thiết cho hệ thống:

```
JWT_SECRET=your_jwt_secret_key_here
GOOGLE_CLIENT_ID=your_google_oauth2_client_id.apps.googleusercontent.com
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_KEY=your-supabase-anon-key
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password
```

4.2.3 Bước 3: Khởi chạy tầng hạ tầng cơ sở (Infrastructure)

Triển khai hệ quản trị CSDL PostgreSQL, Message Broker Kafka, Redis Cache và MinIO Storage trước bằng cách chạy lệnh:

```
docker-compose -f docker-compose.infra.yml up -d
```

Lưu ý quan trọng: Đợi khoảng 10-15 giây để Kafka Broker hoàn thành quá trình bầu chọn Controller và PostgreSQL khởi tạo thành công các slot lưu trữ.

4.2.4 Bước 4: Thực thi di trú cơ sở dữ liệu (Database Migrations)

Chạy container tạm để thực thi việc tạo cấu trúc cơ sở dữ liệu cho toàn bộ các phân hệ bao gồm việc tạo schema, bảng, trigger và các giá trị mặc định ban đầu:

```
docker-compose -f docker-compose.migrate.yml up
```

Đảm bảo tất cả tác vụ di trú chạy thành công và container tự động dừng lại với mã thoát là 0 (exit code 0).

4.2.5 Bước 5: Biên dịch và Khởi chạy các Dịch vụ Nghiệp vụ (Microservices)

Bắt đầu biên dịch mã nguồn các dịch vụ microservices Go/Python và Gateway, khởi chạy ngay:

```
docker-compose -f docker-compose.services.yml up -d --build
```

4.3 Các lưu ý quan trọng và Kinh nghiệm xử lý sự cố (Troubleshooting)

Trong quá trình vận hành, người quản trị hệ thống cần lưu ý một số điểm kỹ thuật quan trọng sau để tránh gặp lỗi bất thường:

4.3.1 Đồng bộ trạng thái khởi động (Service Startup Ordering)

Mặc dù đã sử dụng cơ chế kiểm thử tính sẵn sàng `service_healthy` trong cấu hình Docker Compose để kiểm soát việc khởi động của `identity-service` phụ thuộc vào tính sẵn sàng của PostgreSQL, trong một số trường hợp ổ cứng máy host bị quá tải dẫn đến PostgreSQL phản hồi chậm, dịch vụ `identity-service` có thể bị thoát lỗi trước khi DB sẵn sàng kết nối. Cách khắc phục nhanh nhất là khởi động lại riêng lẻ dịch vụ đó:

```
docker-compose -f docker-compose.services.yml restart identity-service
```

4.3.2 Kích hoạt và Kiểm thử Trang cầu nối Deep Link trên Di động

Ứng dụng di động sử dụng giao thức deep link `pody://` để tiếp nhận liên kết chuyển hướng mở ứng dụng từ email kích hoạt tài khoản hoặc đặt lại mật khẩu. Khi triển khai trên môi trường giả lập (Emulator), base URL trong email được trở về địa chỉ IP cục bộ của máy host (`10.0.2.2:8080` đối với Android hoặc `localhost:8080` đối với iOS) để đảm bảo trình duyệt trên máy ảo có thể phân giải được đường dẫn của Gateway.

4.3.3 Sự cố nghẽn tiến trình Transactional Outbox

Nếu phát hiện người dùng đăng ký thành công tài khoản nhưng hòm thư không nhận được email kích hoạt, cần thực hiện kiểm tra theo hai hướng:

1. Kiểm tra xem bảng `outbox_events` trong Identity Database có bị dồn ứ các bản ghi ở trạng thái `pending` hoặc `failed` hay không. Nếu có, nguyên nhân là tiến trình `OutboxPublisher` ngậm kết nối tới Kafka Broker bị gián đoạn, cần restart container `identity-service`.
2. Nếu bản ghi đã chuyển sang trạng thái `published` nhưng vẫn không có email, cần kiểm tra log của `notification-service` để xem cấu hình tài khoản SMTP gửi mail có bị Google từ chối do sai mật khẩu ứng dụng (App Password) hay không.

4.3.4 Xác thực nhanh trạng thái hoạt động của hệ thống

Sau khi hoàn tất cài đặt, người phát triển có thể kiểm tra nhanh tính sẵn sàng của phân hệ Gateway và Identity thông qua các địa chỉ URL dưới đây:

Bảng 4.1: Bảng URL kiểm thử nhanh hệ thống

Thành phần	Địa chỉ URL cục bộ	Mục đích kiểm tra
Health Check Gateway	http://localhost:8080/healthz	Xác nhận API Gateway hoạt động bình thường.
Tài liệu APIs Gateway	http://localhost:8080/docs	Xem giao diện tài liệu Swagger tích hợp.
Định tuyến Metadata	http://localhost:8080/api/v1/_meta/routes	Kiểm tra danh sách ánh xạ các routes.
Kafka Admin UI	http://localhost:8090	Giám sát các topic và consumer groups.
MinIO Web Console	http://localhost:9001	Xem kho lưu trữ file media và avatar.